МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 4

по дисциплине «Математические основы баз данных»

по теме: «Оконные функции и аналитика данных.»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель и задачи

Цель: освоить применение оконных функций в СУБД PostgreSQL для выполнения сложной аналитики без схлопывания детальных строк. Научиться ранжировать данные, вычислять нарастающие итоги, рассчитывать доли в группах и сравнивать показатели между смежными периодами, а также оценить преимущества оконных функций перед классической группировкой.

Задачи:

1. Ранжирование: Построить рейтинг тарифных планов (или услуг) по стоимости внутри каждой категории с использованием функций ROW_NUMBER, RANK и DENSE_RANK. Продемонстрировать и пояснить разницу в их нумерации на строках с одинаковыми значениями цены.
2. Топ-N в группе: Реализовать выборку трёх самых дорогих услуг в каждой категории, используя оконную нумерацию строк и фильтрацию результатов через общее табличное выражение (CTE).
3. Нарастающий итог: Вычислить кумулятивную (нарастающую) сумму выручки по дням с помощью оконной агрегатной функции SUM() OVER(ORDER BY ...).
4. Доля в группе: Рассчитать долю выручки каждого тарифного плана в общей выручке его категории, применяя конструкцию SUM() OVER(PARTITION BY ...).
5. Сравнение периодов (LAG/LEAD): Применить функции смещения LAG и LEAD для сравнения ежедневной выручки с предыдущим и последующим днями, вычислив абсолютный и процентный прирост (или спад).
6. Сравнение подходов: Решить одну из аналитических задач двумя способами — с помощью оконной функции и через классический GROUP BY с вложенным подзапросом. Обосновать, почему оконный подход в данном случае удобнее и эффективнее.

Выполнение

1. Ранжирование тарифных планов по цене внутри категории подключения

Построить рейтинг тарифных планов по абонентской плате в разрезе типа подключения (Оптика / Ethernet). Запрос демонстрирует три ранжирующие оконные функции и наглядно показывает разницу в их поведении на строках с одинаковыми значениями цены. Для демонстрации эффекта "пропусков" в RANK добавим в таблицу тарифов ещё один план с ценой, совпадающей с существующим.

```
INSERT INTO tariffs (name, speed_down, speed_up, monthly_price,
connection_type)
VALUES ('Комфорт', 100, 50, 600.00, 'Оптика');

SELECT
    connection_type AS category,
    name AS tariff_name,
    monthly_price,
    ROW_NUMBER() OVER w AS row_num,
    RANK() OVER w AS rank_num,
    DENSE_RANK() OVER w AS dense_rank_num
FROM tariffs
WINDOW w AS (PARTITION BY connection_type ORDER BY monthly_price
DESC);
```

	category character varying (50)	tariff_name character varying (100)	monthly_price numeric (10,2)	row_num bigint	rank_num bigint	dense_rank_num bigint
1	Ethernet	Бизнес	1500.00	1	1	1
2	Оптика	Геймер	900.00	1	1	1
3	Оптика	Домашний	600.00	2	2	2
4	Оптика	Комфорт	600.00	3	2	2
5	Оптика	Старт	450.00	4	4	3

2. Выборка Top-N записей в каждой группе через CTE

Отобрать по два самых дорогих тарифных плана для каждого типа подключения (Оптика и Ethernet). Задача решается комбинацией оконной функции ROW_NUMBER() и общего табличного выражения (CTE), что

является стандартным паттерном для получения "топ-N" в реляционных базах данных.

```
WITH ranked_tariffs AS (
    SELECT
        connection_type,
        name AS tariff_name,
        monthly_price,
        ROW_NUMBER() OVER (
            PARTITION BY connection_type
            ORDER BY monthly_price DESC
        ) AS rn
    FROM tariffs
)
SELECT
    connection_type,
    tariff_name,
    monthly_price,
    rn AS rank_in_category
FROM ranked_tariffs
WHERE rn <= 2
ORDER BY connection_type, rn;
```

	connection_type character varying (50)	tariff_name character varying (100)	monthly_price numeric (10,2)	rank_in_category bigint
1	Ethernet	Бизнес	1500.00	1
2	Оптика	Геймер	900.00	1
3	Оптика	Домашний	600.00	2

3. Нарастающий итог выручки по дням

Рассчитать ежедневную выручку провайдера и нарастающий (кумулятивный) итог с начала наблюдаемого периода. Запрос демонстрирует использование агрегатной функции SUM в роли оконной — она вычисляет сумму не по группе строк (как в GROUP BY), а по «окну», которое растёт с каждой следующей строкой, накапливая результат.

```
SELECT
    payment_date::date AS day,
```

```
    SUM(amount) AS daily_revenue,
    SUM(SUM(amount)) OVER (
        ORDER BY payment_date::date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM payments
GROUP BY payment_date::date
ORDER BY day;
```

	day date	daily_revenue numeric	running_total numeric
1	2025-11-...	1500.00	1500.00
2	2025-11-...	1500.00	3000.00
3	2025-11-...	600.00	3600.00
4	2025-11-...	1800.00	5400.00
5	2025-11-...	3600.00	9000.00
6	2025-11-...	1500.00	10500.00
7	2025-11-...	2100.00	12600.00
8	2025-11-...	1500.00	14100.00
9	2025-11-...	6300.00	20400.00
10	2025-11-...	3300.00	23700.00
11	2025-11-...	900.00	24600.00
12	2025-11-...	2250.00	26850.00
13	2025-11-...	1650.00	28500.00

4. Доля выручки тарифа в общем объёме по типу подключения

Рассчитать, какой процент от общей выручки по типу подключения (Оптика / Ethernet) приносит каждый конкретный тарифный план. Запрос демонстрирует использование агрегатной функции SUM в роли оконной с секцией PARTITION BY, что позволяет вычислить сумму по группе, не схлопывая детальные строки результата.

```
WITH tariff_revenue AS (
    SELECT
        t.connection_type,
```

```
        t.name AS tariff_name,
        SUM(p.amount) AS tariff_revenue
    FROM payments p
    JOIN contracts c ON p.contract_id = c.contract_id
    JOIN tariffs t ON c.tariff_id = t.tariff_id
    WHERE c.status = 'active'
    GROUP BY t.connection_type, t.tariff_id, t.name
)
SELECT
    connection_type,
    tariff_name,
    tariff_revenue,
    SUM(tariff_revenue) OVER (PARTITION BY connection_type) AS
type_total_revenue,
    ROUND(
        100.0 * tariff_revenue / NULLIF(SUM(tariff_revenue) OVER
(PARTITION BY connection_type), 0),
        2
    ) AS revenue_share_percent
FROM tariff_revenue
ORDER BY connection_type, revenue_share_percent DESC;
```

	connection_type character varying (50)	tariff_name character varying (100)	tariff_revenue numeric	type_total_revenue numeric	revenue_share_percent numeric
	Ethernet	Бизнес	621000.00	621000.00	100.00
	Оптика	Геймер	693900.00	1353150.00	51.28
	Оптика	Домашний	450000.00	1353150.00	33.26
	Оптика	Старт	209250.00	1353150.00	15.46

5. Сравнение выручки текущего дня с предыдущим и последующим (LAG / LEAD)

Проанализировать динамику ежедневной выручки, сравнив её с показателями предыдущего и следующего дня. Запрос демонстрирует применение функций смещения LAG и LEAD, которые позволяют обращаться к значениям соседних строк в окне без необходимости делать самосоединение (SELF JOIN) таблицы.

```
WITH daily_stats AS (
    SELECT
        payment_date::date AS day,
        SUM(amount) AS daily_revenue
```

```
    FROM payments
    GROUP BY payment_date::date
)
SELECT
    day,
    daily_revenue,
    LAG(daily_revenue, 1) OVER (ORDER BY day) AS prev_day_revenue,
    LEAD(daily_revenue, 1) OVER (ORDER BY day) AS next_day_revenue,
    daily_revenue - LAG(daily_revenue, 1) OVER (ORDER BY day) AS
absolute_growth,
    ROUND(
        100.0 * (daily_revenue - LAG(daily_revenue, 1) OVER (ORDER BY day))
        / NULLIF(LAG(daily_revenue, 1) OVER (ORDER BY day), 0),
        2
    ) AS growth_percent
FROM daily_stats
ORDER BY day
LIMIT 10;
```

	day date	daily_revenue numeric	prev_day_revenue numeric	next_day_revenue numeric	absolute_growth numeric	growth_percent numeric
1	2025-11-...	1500.00	[null]	1500.00	[null]	[null]
2	2025-11-...	1500.00	1500.00	600.00	0.00	0.00
3	2025-11-...	600.00	1500.00	1800.00	-900.00	-60.00
4	2025-11-...	1800.00	600.00	3600.00	1200.00	200.00
5	2025-11-...	3600.00	1800.00	1500.00	1800.00	100.00
6	2025-11-...	1500.00	3600.00	2100.00	-2100.00	-58.33
7	2025-11-...	2100.00	1500.00	1500.00	600.00	40.00
8	2025-11-...	1500.00	2100.00	6300.00	-600.00	-28.57
9	2025-11-...	6300.00	1500.00	3300.00	4800.00	320.00
10	2025-11-...	3300.00	6300.00	900.00	-3000.00	-47.62

6. Сравнение оконной функции и GROUP BY на одной задаче

Решить одну и ту же аналитическую задачу двумя способами - через оконную функцию и через классическую группировку с вложенным подзапросом - и наглядно показать, почему оконный подход даёт более читаемый, короткий и эффективный код. В качестве задачи выбрана доля выручки каждого тарифа в общей выручке провайдера (аналог Задания 4, но в более простом виде - без разбивки по типу подключения).

Оконная функция

```
WITH tariff_revenue AS (
    SELECT
        t.name AS tariff_name,
        SUM(p.amount) AS revenue
    FROM payments p
    JOIN contracts c ON p.contract_id = c.contract_id
    JOIN tariffs t ON c.tariff_id = t.tariff_id
    WHERE c.status = 'active'
    GROUP BY t.tariff_id, t.name
)
SELECT
    tariff_name,
    revenue,
    SUM(revenue) OVER () AS total_revenue,
    ROUND(100.0 * revenue / SUM(revenue) OVER (), 2) AS share_percent
FROM tariff_revenue
ORDER BY revenue DESC;
```

	tariff_name character varying (100)	revenue numeric	total_revenue numeric	share_percent numeric
1	Геймер	693900.00	1974150.00	35.15
2	Бизнес	621000.00	1974150.00	31.46
3	Домашний	450000.00	1974150.00	22.79
4	Старт	209250.00	1974150.00	10.60

GROUP BY с вложенным подзапросом

```
WITH tariff_revenue AS (
    SELECT
        t.name AS tariff_name,
        SUM(p.amount) AS revenue
    FROM payments p
    JOIN contracts c ON p.contract_id = c.contract_id
    JOIN tariffs t ON c.tariff_id = t.tariff_id
    WHERE c.status = 'active'
    GROUP BY t.tariff_id, t.name
),
total AS (
    SELECT SUM(revenue) AS total_revenue FROM tariff_revenue
```

```
)
SELECT
    tr.tariff_name,
    tr.revenue,
    t.total_revenue,
    ROUND(100.0 * tr.revenue / t.total_revenue, 2) AS share_percent
FROM tariff_revenue tr
CROSS JOIN total t
ORDER BY tr.revenue DESC;
```

	tariff_name character varying (100)	revenue numeric	total_revenue numeric	share_percent numeric
1	Геймер	693900.00	1974150.00	35.15
2	Бизнес	621000.00	1974150.00	31.46
3	Домашний	450000.00	1974150.00	22.79
4	Старт	209250.00	1974150.00	10.60

Вывод

В ходе выполнения лабораторной работы №4 были освоены оконные функции СУБД PostgreSQL и их применение для решения аналитических задач без схлопывания детальных строк. База данных интернет-провайдера, спроектированная и наполненная в предыдущих работах, послужила основой для проведения многомерной аналитики тарифных планов, платежей и подключённых услуг.